System Logowania

Django — Widoki POST & Uwierzytelnianie

Etap 1 — Walidacja formularza

Co sprawdzamy?

- Czy dane logowania sa **poprawnie przekazane** (metoda POST)?
- Czy wymagane pola **nie sa puste**?
- Czy logowanie mozna uznac za **poprawne** (format, dlugosc itp.)?

Etap 2 — Uwierzytelnianie uzytkownika

Walidacja przeszla pomyslnie — sprawdzamy teraz, czy podane dane **odpowiadaja rzeczywistemu uzytkownikowi** w bazie danych.

Logowanie OK

- Zapisuje `id` usera w **sesji**
- Kolejne requesty maja info o userze
- W **cookie** przechowywany jest *tylko identyfikator sesji* (nie dane usera)

Logowanie FAILED

- Nikogo nie loguje
- Wyswietla odpowiedni komunikat bledu
- Ponownie renderuje formularz

Cel systemu

Do czego to sluzy?

Zalogowany **student** moze:

- **Zapisac sie** na zajecia
- **Wypisac sie** z zajec

Mechanizm chroni **system zapisywania sie na zajecia** przed nieautoryzowanym dostepem.

Widoki: GET vs POST

Metoda	Zastosowanie
GET	Wczesniejsze widoki — **pobieranie i wyswietlanie** danych bez modyfikacji stanu
POST	Obecne widoki — **przesylanie** danych z formularza, modyfikowanie stanu

Pomocnicza funkcja: `_current_student()`

```
def _current_student(request) -> Student | None:
    return getattr(request.user, "student_profile", None)
```

Union Type `Student | None` — funkcja zwraca albo obiekt `Student`, albo `None`. Typ zwracany jest jawnie zadeklarowany (Python 3.10+).

`getattr(obj, name, default)` — bezpieczna obsluga braku atrybutu. Gdy `request.user` nie ma atrybutu `student_profile`, zamiast rzucic `AttributeError`, funkcja zwraca `None`.

Dwuwarstwowa ochrona widoku

Warstwa 1 — `@login_required`

```
@login_required()
def moj_widok(request):
    ...
```

Dekorator sprawia, ze widok jest dostepny **wylacznie dla zalogowanych** uzytkownikow. Niezalogowany uzytkownik jest **przekierowywany** na strone logowania.

Warstwa 2 — Sprawdzenie profilu studenta

```
student = _current_student(request)    # Student albo None
if student is None:
    # brak profilu studenta -> brak dostepu
    return redirect("brak_uprawnien")
```

Sprawdza, czy zalogowane konto **ma profil studenta**. Samo bycie zalogowanym nie wystarczy.

Dlaczego dwie warstwy?

1. **@login_required** — odfiltrowuje *anonimowych* uzytkownikow
2. **_current_student()** — odfiltrowuje zalogowanych *bez profilu studenta* (np. adminow, wykladowcow)

Tylko **zalogowany student** przechodzi obie warstwy i moze korzystac z systemu.

Transakcja atomowa: `atomic()`

```
from django.db import transaction

with transaction.atomic():
    zajecia.studenci.add(student)
    # jesli cos sie nie uda -> ROLLBACK calego bloku
```

Zasada: *wszystko albo nic.* Jesli ktoras operacja w bloku sie nie powiedzie, **cala transakcja jest wycofana (rollback)** — baza danych pozostaje w spojnym stanie.

Uwaga: `GET` uzywamy gdy chcemy cos *pokazac* ale nie zmieniac stanu. Do zapisu uzywamy `POST` + `atomic()`.
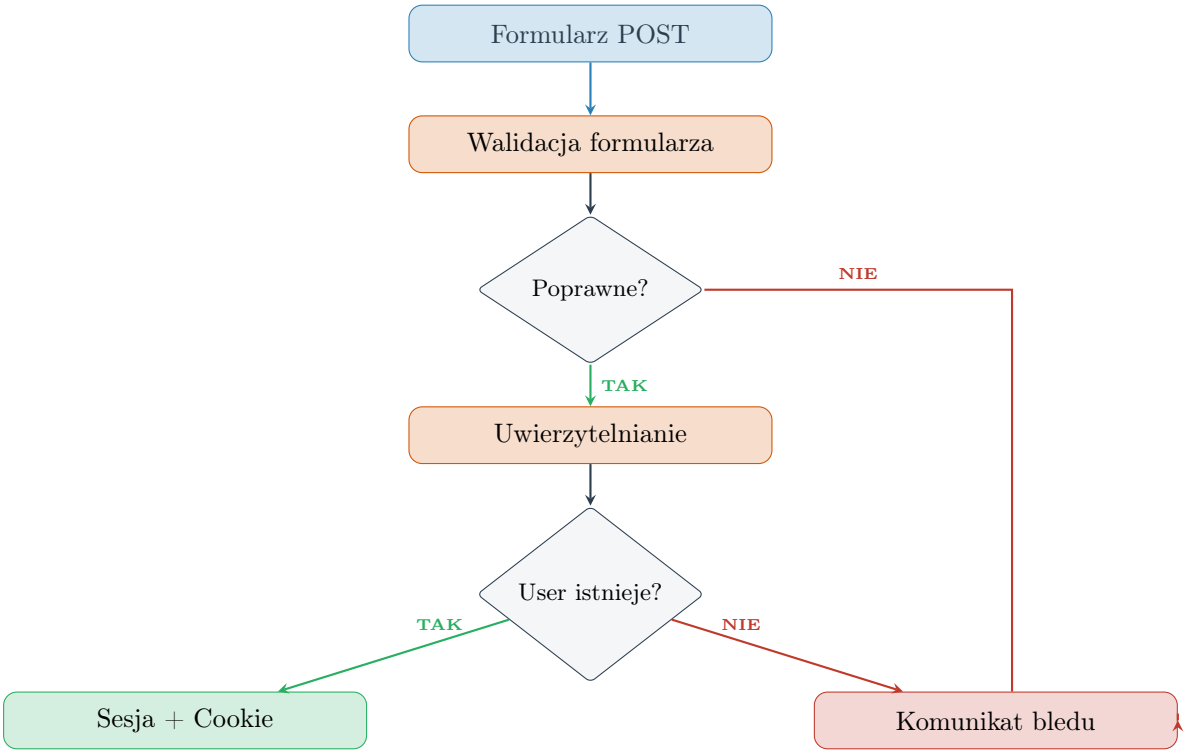
Diagram: przeplyw logowania